MP GROUP 3

BUAN, HIRAYA SABATER
ESTRELLA, ELMERANITA DOLERO
HERRERA, RAPHAEL ALFONSO MOLETA

MCO2 REPORT

I. Introduction

The goal of this project is to create a comprehensive social network management system that represents user connections using a matrix as the underlying data structure. By loading information from an external file, displaying friends lists for a certain user ID, and quickly looking for links between two user IDs, the program enables users to interact with the social network. By giving users a continuous social network relationship exploration experience, this seeks to improve social analysis and comprehension.

II. Data Structure

The data structure that was used in representing the social graph data is an adjacency matrix. This matrix uses a 2D array where the cell 'i' and 'j' represents the connection between the nodes 'i' and 'j' in the graph. In the representation, the value of 1 in cell i and j indicates that there is an edge between node i and node j, while a value of 0 shows no edge.

The reason for using the adjacency matrix is its simplicity and ease of implementation make it an accessible data structure, particularly for small to moderately sized graphs. Secondly it efficiently handles dense graphs, where there are many connections between nodes. As an adjacency matrix uses a fixed amount of memory, it avoids the overhead of dynamic memory allocation and can be more efficient than other data structures like adjacency lists in such cases. The adjacency matrix allows for quick edge existence checking between two nodes. This constant time complexity (O(1)) for checking whether an edge exists is valuable for certain graph algorithms, especially those that involve frequent edge-related operations. Lastly, for small graphs or situations where the graph structure remains relatively static, the overhead of using an adjacency matrix is acceptable. In these scenarios, the advantages of constant-time edge checking and straightforward implementation outweigh any potential drawbacks, making it a pragmatic choice.

III. Algorithms

The program starts by reading the data from a file provided by the user. The file should contain the number of nodes (n) and the number of edges (e) representing the connections in the social network. It allocates memory for a 2D matrix (Records) to store the connections between individuals. Each cell (i, j) in the matrix is set to 1 if there is a connection between IDs i and j, and 0 otherwise.

The user is presented with a main menu and can choose between the two actions mentioned above. Depending on the user's choice, the program will either display the list of friends connected to a given ID (getfriendsList() function) or check if there is a direct connection between two IDs (checkConnection() function). The program handles input validation to ensure the user provides valid IDs and handles incorrect inputs.

The program measures the execution time for each action using the currentTimeMillis() function, which uses the gettimeofday() function to get the current time in milliseconds. After the user completes an action, the program will return to the main menu until the user chooses to exit the program.

Finally, the program frees the dynamically allocated memory for the 2D matrix before exiting.

IV. Analysis

In order to record and calculate the time, the function currentTimeMillis was implemented in order to get the exact milliseconds.

```
long currentTimeMillis() {
    struct timeval time;
    gettimeofday(&time, NULL);
    long s1 = (long)(time.tv_sec) * 1000;
    long s2 = (time.tv_usec / 1000);
    return s1 + s2;
}
```

a.) getfriendsList function

```c
int getfriendsList(int n, int **Records) {
    int ID, valid, nFriends;

    nFriends = 0;
    valid = 0;

    do {
        valid = 0;
        printf("Enter ID: ");
        scanf("%d", &ID);

        if (ID >= n || ID < 0) {
            printf("ID number does not exist. Try again.\n\n");
            valid = 1;
        }

    } while (valid == 1);

    printf("\nFRIENDS:\n");

    for (int i = 0; i < n; i++) {
        if (Records[ID][i] == 1) {
            printf("%d\n", i);
            nFriends++;
        }
    }

    printf("\nTotal number of friends: %d\n\n", nFriends);

    return 1;
}
```

Time Complexity: O(n)

In order to calculate the execution time of the function, this block of code was added to the main function:

```c
    startTime = currentTimeMillis();
    getfriendsList(n, Records);
    endTime = currentTimeMillis();
    printf("Execution Time: %ldms\n\n", endTime - startTime);
    valid = 1;
```

Results:

Data Used	***1st Test***	***2nd Test***	***3rd Test***	***Average***
Caltech36	3400 ms	2085 ms	2200 ms	2562 ms
Harvard1	14415 ms	5889 ms	11286 ms	10530 ms
Rice31	18776 ms	2884 ms	5473 ms	9044 ms
Stanford3	69699 ms	9051 ms	2732 ms	27161 ms
Trinity100	7577 ms	5038 ms	2069 ms	4895 ms

ms = milliseconds

b.) checkConnection function

```c
int checkConnection(int n, int **Records) {
    int ID1, ID2, valid;

    valid = 0;

    printf("\nCheck connection of ID numbers:\n");

    do {
        valid = 0;
        printf("\nEnter ID 1: ");
        scanf("%d", &ID1);

        if (ID1 >= n || ID1 < 0) {
            printf("ID not in dataset. Please try again.\n\n");
            valid = 1;
        } else {
            printf("\nEnter ID 2: ");
            scanf("%d", &ID2);

            if (ID2 >= n || ID2 < 0) {
                printf("ID not in dataset. Please try again.\n\n");
                valid = 1;
            } else {
                if (ID1 == ID2) {
                    printf("Both IDs are the same. No connection is needed.\n\n");
                } else {
                    if (Records[ID1][ID2] == 1) {
                        printf("A connection was found.\n\n");
                    } else {
                        printf("A connection was not found.\n\n");
                    }
                }
            }
        }
    } while (valid);

    return 1;
}
```

Time Complexity: O(1)

In order to calculate the execution time of the function, this block of code was added to the main function:

```c
startTime = currentTimeMillis();
checkConnection(n, Records);
endTime = currentTimeMillis();
printf("Execution Time: %ldms\n\n", endTime - startTime);
valid = 1;
```

Results:

Data Used	***1st Test***	***2nd Test***	***3rd Test***	***Average***
Caltech36	19437 ms	9427 ms	8162 ms	12342 ms
Harvard1	15159 ms	8860 ms	9481 ms	11167 ms
Rice31	15894 ms	9271 ms	7021 ms	10729 ms
Stanford3	16712 ms	8013 ms	9257 ms	11327 ms
Trinity100	17301 ms	7635 ms	8689 ms	11208 ms

ms = milliseconds

V. Summary

To summarize, to create a fast and efficient social network management system, a 2D adjacency matrix was used. The time complexity for the first algorithm is dependent on the size of the data while the time complexity for the second algorithm is constant regardless of the input size. By implementing both of these concepts, the program is able to quickly determine the connection between two users and the number of friends for a specific user.

Group Contribution

BUAN, HIRAYA SABATER	Coding, Debugging, Writing of Analysis and Summary in report
ESTRELLA, ELMERANITA DOLERO	Coding, Debugging, Writing of Algorithm in report
HERRERA, RAPHAEL ALFONSO MOLETA	Coding, Debugging, Writing of Introduction and Data Structure in report